

browser, interprets and runs the Selenese commands passed from the test program, and acts as an HTTP proxy, intercepting and verifying HTTP messages passed between the browser and Application Under Test (AUT).

- Client libraries that provide the interface between each programming language and the Selenium RC server.

Selenium RC is great for testing complex AJAX based Web user interfaces under a continuous integration system. It is also an ideal solution for users of Selenium IDE who want to write tests in a more expressive programming language than the Selenese HTML table format.

Selenese commands

Selenese is the set of Selenium commands which is used to test Web applications. The tester can test the broken links, the existence of some object on the UI, AJAX functionality, the alert window, list options and a lot more using Selenese. There are three types of commands:

- Actions:** These are commands that manipulate the state of the application. Upon execution, if an action fails, the execution of the current test is stopped. Some examples are: *click()*: Clicks on a link, button, checkbox or radio button. *contextMenuAt (locator, coordString)*: Simulates the user by clicking the 'Close' button in the title bar of a popup window or tab.
- Accessors:** These evaluate the state of the application and store the results in variables which are used in assertions. Some examples are: *assertErrorOnNext*: Pings Selenium to expect an error on the next command execution with an expected message. *storeAllButtons*: Returns the IDs of all buttons on the page.
- Assertions:** These enable us to verify the state of an application and compare it against the expected. It is used in three modes, i.e., assert, verify and waitfor. Some examples are: *waitForErrorOnNext(message)*: Wait for error, used with the accessor *assertErrorOnNext*. *verifySelected (selectLocator, opt onLocator)*: Verifies that the selected item of a drop-down satisfies *optionSpecifier*.

Selenium WebDriver

Selenium WebDriver is a tool that automates the testing of Web applications and is popularly known as Selenium 2.0. It is a Web automation framework that allows you to execute your tests against different browsers. WebDriver also enables you to use a programming language in creating your test scripts. The following programming languages are supported by Selenium WebDriver:

- Java
- .NET
- PHP
- Python
- Perl
- Ruby

The WebDriver uses a different underlying framework, while Selenium RC uses a JavaScript Selenium-Core embedded within the browser, which has its limitations. WebDriver directly interacts with the browser without any intermediary. Selenium RC depends on a server.

Architecture

The architecture of WebDriver is explained in Figure 1.

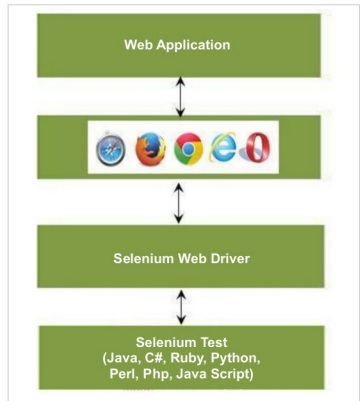


Figure 1: Architecture of Selenium WebDriver

The differences between WebDriver and Selenium RC are given in Table 1.

Table 1

WebDriver	Selenium RC
Architecture is simpler, as it controls the browser from the OS level.	Architecture is complex, as it depends on the server.
It supports HtmlUnit.	It does not support HtmlUnit.
WebDriver is faster, as it interacts directly with the browser.	It is slower, as it uses JavaScript to interact with RC.
Less object-oriented APIs and cannot be used for mobile testing.	Purely object-oriented and can be used for iPhone/Android application testing.
WebDriver is not ready to support new browsers and does not have a built-in command for the automatic generation of test results.	Selenium RC can support new browsers and have built-in commands.

Continued to page 64....